Department of Software Engineering

Cloud Computing (SE-490) - Complex Engineering Activity (CEA)

Design, Deployment and Cloud-Native Enhancement of a Three-Tier Web Application

Technical Report - Personal Expense Tracker on Amazon Web Services

Submitted by	Abdur Rafey
Cloud provider	Amazon Web Services (AWS), region ap-southeast-2 (Sydney)
Live application	http://expense-tracker-public-alb-1866684008.ap-southeast-2.elb.amazonaws.com
Source repository	https://github.com/abdurrafey250904-hash/Expense-Tracker
CI/CD	GitHub Actions (`.github/workflows/ci-cd.yml`)
Infrastructure as Code	Terraform (`infra/terraform/`)
Date	November 2026

1. Introduction and Choice of Cloud Provider

The application is a full-stack **Personal Expense Tracker** (MERN stack): users register and log in (JWT), record categorized expenses, set monthly and per-category budgets with alerts, view charts and monthly reports, and export reports as PDF/CSV. It is naturally a **three-tier application**: a React single-page application (presentation tier), an Express/Node.js REST API (application/business-logic tier) and MongoDB (data tier). This report describes how it was containerized, deployed on AWS across public and private network tiers, and automated with a CI/CD pipeline, and justifies the design against the required non-functional requirements (NFRs).

Why AWS. AWS was selected because (i) it offers every building block the brief asks for as first-class managed services — VPC, NAT Gateway, Application Load Balancers with target groups, EC2 Auto Scaling Groups, Elastic Container Registry, Secrets Manager, Systems Manager, S3, EBS and CloudWatch; (ii) its Free plan ($100 credits, free-tier-eligible `t3.micro/t3.small` instances) allowed a realistic multi-AZ deployment at no personal cost; (iii) Terraform's AWS provider and GitHub Actions' official AWS actions are mature, enabling a fully reproducible infrastructure-as-code and CI/CD workflow; and (iv) it is the most widely used provider in industry, so the skills transfer directly.

> **Account constraints discovered and handled.** The AWS Free-plan account is governed by an organization Service Control Policy that (a) allows only the **ap-southeast-2 (Sydney)** region, (b) denies Amazon DocumentDB / RDS cluster and instance creation, and (c) denies creating an IAM OIDC identity provider. The design was therefore deployed in Sydney, uses the brief's permitted alternative of a *properly secured self-managed database instance*, and authenticates CI/CD with a narrowly scoped IAM user instead of OIDC. Each constraint is documented as a limitation in Section 7.

2. Overall Architecture and Design Decisions

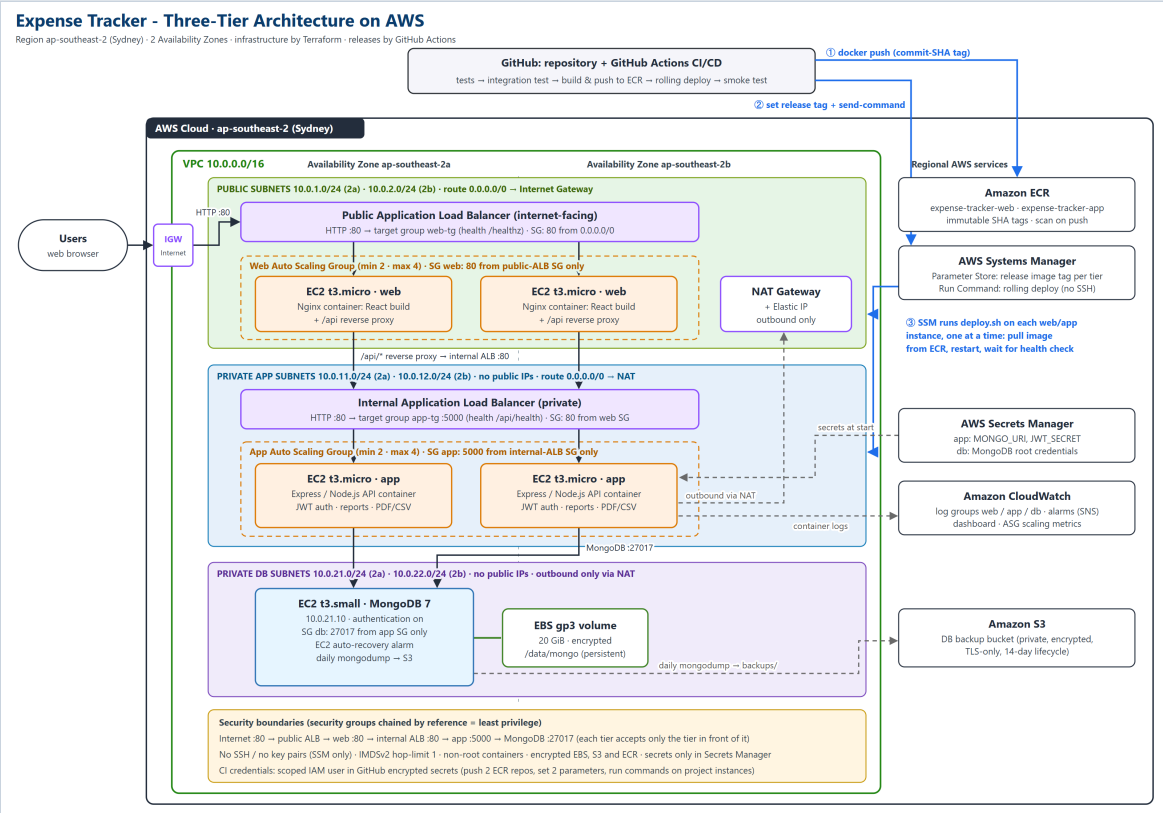


Figure 1 - Architecture: VPC with public, private-app and private-DB subnets in two Availability Zones, two load balancers, two Auto Scaling Groups, the MongoDB host, and the GitHub Actions → ECR → SSM deployment flow. Full-resolution version submitted separately as `Architecture-Diagram.png`.

Request path. Browser → **public Application Load Balancer** (internet-facing, public subnets) → **web tier** (Nginx container serving the React build) → for `/api/*` Nginx reverse-proxies to the **internal ALB** (private subnets, not

internet-reachable) → **application tier** (Express container) → **MongoDB** on a dedicated host in the private DB subnet.

2.1 Network design

Component	Configuration	Purpose
VPC	`10.0.0.0/16`, DNS hostnames on, 2 AZs (ap-southeast-2a/2b)	Isolated private address space for all tiers
Public subnets	`10.0.1.0/24`, `10.0.2.0/24`	Public ALB, NAT Gateway, web-tier instances
Private app subnets	`10.0.11.0/24`, `10.0.12.0/24`	Internal ALB and application-tier instances (no public IPs)
Private DB subnets	`10.0.21.0/24`, `10.0.22.0/24`	MongoDB host (no public IP)
Internet Gateway	attached to the VPC	Internet connectivity for public subnets
NAT Gateway	in public subnet 2a with an Elastic IP	Outbound-only internet for private hosts (image pulls, patches, Secrets Manager, S3)
Route tables	public: `0.0.0.0/0 → IGW`; private-app and private-db: `0.0.0.0/0 → NAT`	Private hosts can initiate outbound traffic but nothing can connect in

2.2 Key design decisions

- **EC2 Auto Scaling Groups running Docker** (instead of ECS/EKS): matches the brief's "Auto Scaling Groups" and "pull the new image onto the target compute instance", and keeps each tier's compute, scaling and security group independent.
- **Two load balancers**: the public ALB fronts the web tier; a separate *internal* ALB fronts the API so that the application tier is reachable *only* from the web tier, as the brief requires.
- **Same-origin API**: the browser calls `/api` on the same host; the web tier proxies it. The API is never exposed to the internet and no CORS surface is needed.
- **Release pointer in SSM Parameter Store**: the pipeline publishes the image tag (git SHA); instances read it at boot, so instances launched later by auto scaling always run the current release.
- **No SSH anywhere**: deployment and administration use AWS Systems Manager Run Command / Session Manager; no key pairs or port 22 rules exist.
- **Everything as code**: about 100 AWS resources are defined in Terraform and can be recreated or destroyed with one command.

3. Services Used and Their Role in Each Tier

Tier / area	AWS service	Role in this deployment
Web tier	EC2 Auto Scaling Group `expense-tracker-web-asg` (t3.micro × 2-4), public ALB + target group `web-tg`	Runs the `expense-tracker-web` container (unprivileged Nginx + React build) on port 80; ALB health check `/healthz`
Application tier	EC2 Auto Scaling Group `expense-tracker-app-asg` (t3.micro × 2-4), internal ALB + target group `app-tg`	Runs the `expense-tracker-app` container (Express API, port 5000); health check `/api/health` reports DB state and release version
Data tier	EC2 `expense-tracker-mongodb` (t3.small, fixed IP 10.0.21.10), EBS gp3 20 GiB encrypted data volume, S3 backup bucket	MongoDB 7 container with authentication; data on a separate persistent volume; daily compressed `mongodump` to S3 (14-day lifecycle)
Registry	Amazon ECR (2 private repositories)	Stores web and app images; immutable SHA tags, scan-on-push, keep-last-10 lifecycle policy

Secrets	AWS Secrets Manager (`expense-tracker/app`, `expense-tracker/db`)	Generated DB credentials, connection string and JWT secret injected at container start
Deployment	SSM Parameter Store, SSM Run Command	Release tag per tier; rolling, SSH-less deploys from the pipeline
Monitoring	CloudWatch Logs, Alarms, Dashboard, SNS; EC2 auto-recovery	Container logs per tier, 5xx/unhealthy-host/CPU alarms, dashboard, automatic recovery of the DB host
Identity	IAM roles per tier, CI IAM user	Least-privilege access for each tier and for the pipeline
Automation	Terraform, GitHub Actions	Infrastructure as code; build-test-deploy pipeline

4. Containerization and CI/CD Pipeline Design

4.1 Containerization

- **Web image** (`frontend/Dockerfile`): multi-stage — stage 1 builds the React app with Vite on `node:20-alpine`; stage 2 copies only the static files into `nginx-unprivileged:1.27-alpine` (non-root, port 8080). The Nginx config is a template whose API upstream and DNS resolver are injected per environment, and adds SPA routing, gzip, long-term asset caching, security headers and a `/healthz` endpoint.
- **App image** (`backend/Dockerfile`): multi-stage — production-only dependencies installed with `npm ci --omit=dev`, runs as the non-root `node` user, with a Docker `HEALTHCHECK`. The backend was refactored into `app.js` / `server.js` for testability and graceful SIGTERM shutdown.
- **docker-compose.yml** runs web, app and MongoDB locally on two separate Docker networks (web cannot reach the DB network), mirroring the AWS segmentation; the same compose file is used by the pipeline's integration test.
- `.dockerignore` files keep `node_modules`, tests and any `.env` secrets out of the images.

4.2 CI/CD pipeline (GitHub Actions)

#	Job	What it does
1	Backend unit tests	`npm ci && npm test` (Node test runner + Supertest: health, 404, auth guard)
2	Frontend build	Compiles the React app to catch build errors
3	Terraform fmt & validate	Rejects malformed infrastructure code
4	Integration test	`docker compose up --build --wait`, then through Nginx: register a user → create an expense → list it; proves all three containers work together
5-6	Build & push (web, app)	Docker Buildx with layer cache; push to ECR tagged with the commit SHA
7	Rolling deploy	Publishes the SHA to SSM, then runs the deploy script on each in-service instance *one at a time* via SSM Run Command (app tier first, then web) and waits for each container to pass its health check; finally smoke-tests the public URL until `/api/health` returns the new SHA and `db: connected`

Jobs 1-4 run on every push and pull request; jobs 5-7 run only on `main`. Concurrency control prevents overlapping deployments, and documentation-only commits skip the pipeline. Because each release is an immutable SHA-tagged image, rolling back means re-running the pipeline for an earlier commit.

5. Security Considerations

Control	Implementation and justification
Security groups per tier (least privilege)	Five security groups chained by *security-group references*, not IP ranges: public ALB ← internet :80; web ← public-ALB SG :80; internal ALB ← web SG :80; app ← internal-ALB SG :5000; MongoDB ← app SG :27017. Each tier accepts traffic only from the tier directly in front of it.

Network segmentation	Separate public, private-app and private-DB subnets. App and DB hosts have no public IP and reach the internet only outbound through NAT. The internal ALB has no public address (resolves to 10.0.x.x only).
Secrets management	DB password and JWT secret are generated by Terraform and stored only in Secrets Manager; containers receive them at start into `chmod 600` env files. Nothing is hard-coded in code or images; `.env`, `*.tfvars` and Terraform state are git-ignored.
CI/CD secrets	The pipeline authenticates with a dedicated IAM user whose key is stored as **GitHub encrypted secrets** (created and piped directly into GitHub, never displayed). Its policy only allows pushing to the two ECR repositories, updating the two release parameters and sending commands to instances tagged `Project=expense-tracker`.
IAM least privilege	Per-tier instance roles: each can pull only its own image, read only its own release tag and write only its own log group; only the app role may read the app secret; the DB role may only write to `backups/` in its bucket.
Encryption	All EBS volumes (including DB data) encrypted at rest; S3 backups encrypted with public access blocked and a TLS-only bucket policy; ECR encrypted (AES-256).
Host and container hardening	No SSH or key pairs; IMDSv2 required with hop limit 1, so containers cannot obtain instance-role credentials; web and app containers run as non-root; MongoDB authentication enforced.
Supply chain	Immutable image tags, ECR scan-on-push, and tests + integration test gating every deployment.

Verified from the internet (evidence file `10-security-and-backup-tests.txt`): the public ALB returns HTTP 200; a direct request to a web instance's public IP **times out** (blocked by its SG); the internal ALB resolves only to private addresses; the app and DB instances have **no public IP**.

6. Testing Results - Successful Automated Deployment

Test	Result
Backend unit tests (4 tests: health, 404, missing token, invalid token)	**4/4 passed (locally and in CI)**
Integration test: three containers via docker compose (register → add expense → list through Nginx)	**Passed in every pipeline run**
Pipeline runs on `main` (runs #2, #3, #4)	**All 7 jobs green; rolling deploy ~55 s; total ~2.5 min per run**
Rolling deployment	**Each instance reported "Healthy: <ECR image>:<SHA>" before the next was updated; site stayed available**
Post-deploy smoke test through the public ALB	**`/api/health` → `{"status":"ok","db":"connected","version":"db62666..."}`**
Load balancer target health	**web-tg 2/2 healthy, app-tg 2/2 healthy; both ASGs "At desired capacity"**
Database persistence and backup	**Data survives redeploys; manual `mongodump` uploaded to S3 (`backups/*.archive.gz`)**
End-to-end in a browser	**Registration, expenses, dashboard charts, reports and budget alerts work (Figures A21-A26)**

Issue found and fixed by the pipeline. The very first run failed only at the web-tier deploy step: the Auto Scaling Group had just replaced an instance that was not yet registered with Systems Manager. The deploy script was changed to skip still-booting instances (they self-deploy from the published tag at boot) and the CI user was granted `ssm:DescribeInstanceInformation`; every subsequent run has been fully green. This demonstrates the self-healing release-pointer design. The successful pipeline run is shown in Figure A20 (Appendix A.7).

7. Non-Functional Requirements Justification

NFR	Design / implementation decisions that support it	Known limitations and improvements

Scalability	Stateless containers (JWT, no server sessions) behind load balancers; web and app tiers scale horizontally and independently in their own ASGs (2→4 instances); the DB tier is isolated so it can be scaled separately.	Max 4 per tier to protect free credits (one variable to raise). Database scales vertically only → a MongoDB replica set / sharding or a managed cluster would scale reads and writes.
Availability / Reliability	Web and app tiers span **two AZs** with at least two instances each; ELB health checks make the ASGs replace failed instances automatically; Docker restart policies; DB host has EC2 auto-recovery, data on a separate EBS volume and daily S3 backups.	**Single DB instance in one AZ is the main single point of failure** → 3-node replica set across AZs or DocumentDB (blocked on this account). Single NAT Gateway → one per AZ.
Elasticity	Target-tracking policies scale out and back in automatically: average CPU 50 % (both tiers) and 1,000 ALB requests per target (web tier).	Scale-out takes ~2-3 min (boot + image pull); a pre-baked AMI or Fargate would react faster.
Security	Section 5: SG chaining, private subnets, least-privilege IAM, Secrets Manager, encryption at rest, IMDSv2, no SSH, scanned immutable images, scoped CI credentials.	HTTP only (no domain for an ACM certificate) → Route 53 + ACM + HTTPS listener and AWS WAF. App→DB traffic is VPC-internal but not TLS. CI uses an access key because OIDC is blocked.
Maintainability / Deployability	All infrastructure in Terraform; every push runs tests, an integration test, image build, rolling deploy and smoke test automatically; releases are SHA-tagged and roll back by redeploying an older commit; zero manual server access.	Terraform is applied manually with local state → move state to S3 + DynamoDB locking and add a plan/apply stage to the pipeline.
Performance / Latency	Nginx serves static assets with gzip and one-year immutable caching; load is spread by ALBs; all tiers are in one region and VPC; gp3 storage for the DB; MongoDB indexes on `user+date` and `user+category`.	Add CloudFront in front of the web tier for global users and a cache (ElastiCache) for report aggregations.
Cost-Efficiency	Free-tier-eligible t3.micro/t3.small instances; ASGs idle at the minimum of 2; a self-managed DB instead of a ~$70/month managed cluster; one NAT Gateway; ECR lifecycle and 14-day log retention; whole stack destroyable with one command.	See Section 8 for further measures.
Portability	All three tiers are standard Docker images (`nginx-unprivileged`, `node`, `mongo:7`) configured only through environment variables; the same images run locally with docker compose, on EC2 here, and could run unchanged on ECS, EKS, AKS or GKE; switching to Atlas or DocumentDB only changes `MONGO_URI`.	Terraform code is AWS-specific and would need rewriting for another provider.

8. Cost Estimate and Optimization

Service	Configuration (ap-southeast-2, on-demand)	≈ USD / month
EC2 (web + app)	4 × t3.micro, 730 h	38.50
EBS	4 × 10 GB gp3 root volumes	3.85
Application Load Balancers	2 ALBs (public + internal), low LCU usage	48.00
NAT Gateway	1 × 730 h + ~5 GB processed	43.40
Public IPv4 addresses	NAT EIP + 2 web instances + 2 ALB nodes	18.25
Database host	1 × t3.small + 10 GB root + 20 GB data gp3	22.50
S3, ECR, Secrets Manager, CloudWatch, SNS, SSM	backups < 1 GB, 2 secrets, logs/alarms/dashboard	6.05
Total		**≈ 180 / month (≈ 6 / day)**

The deployment runs on AWS Free-plan credits ($100), so the evaluation period costs nothing out of pocket. Figures are on-demand estimates and should be confirmed with the AWS Pricing Calculator.

Cost optimization measures:

- **Replace the NAT Gateway with VPC endpoints** (free S3 gateway endpoint plus ECR, SSM, Secrets Manager and CloudWatch Logs interface endpoints) or a NAT instance, removing most of the ~$43/month NAT charge.
- **Serve the React build from S3 + CloudFront** (or use a single ALB with path routing), removing the two web instances, their public IPs and one ALB (~$50/month).
- **Schedule non-production environments**: scale ASGs to zero and stop the DB host outside working hours, or `terraform destroy` when idle.
- **Graviton and commitments**: t4g instances are ~20 % cheaper (multi-arch images), and Savings Plans cut steady-state compute by 30-40 %.

9. Conclusion

The Expense Tracker was successfully containerized and deployed on AWS as a secure three-tier architecture: a public web tier and a private application tier, each in its own Auto Scaling Group behind its own load balancer across two Availability Zones, and an isolated, authenticated, encrypted and backed-up database tier. Container images are stored in Amazon ECR, and a GitHub Actions pipeline tests, builds, pushes and performs zero-SSH rolling deployments on every change, verified by an automatic smoke test. Five cloud-native enhancements were implemented (S3 object storage, persistent EBS block storage, CloudWatch logging and monitoring, Terraform infrastructure as code and a cost estimate). The main remaining limitations — a single-instance database, HTTP-only access and key-based CI credentials — are consequences of the Free-plan account restrictions and are documented above with concrete improvements.

Appendix A - Deployment Evidence (Screenshots)

All AWS Console screenshots were taken in region ap-southeast-2 (Sydney). Additional full-detail AWS CLI exports are in the repository under `docs/evidence/`.

A.1 Network / VPC configuration and subnets

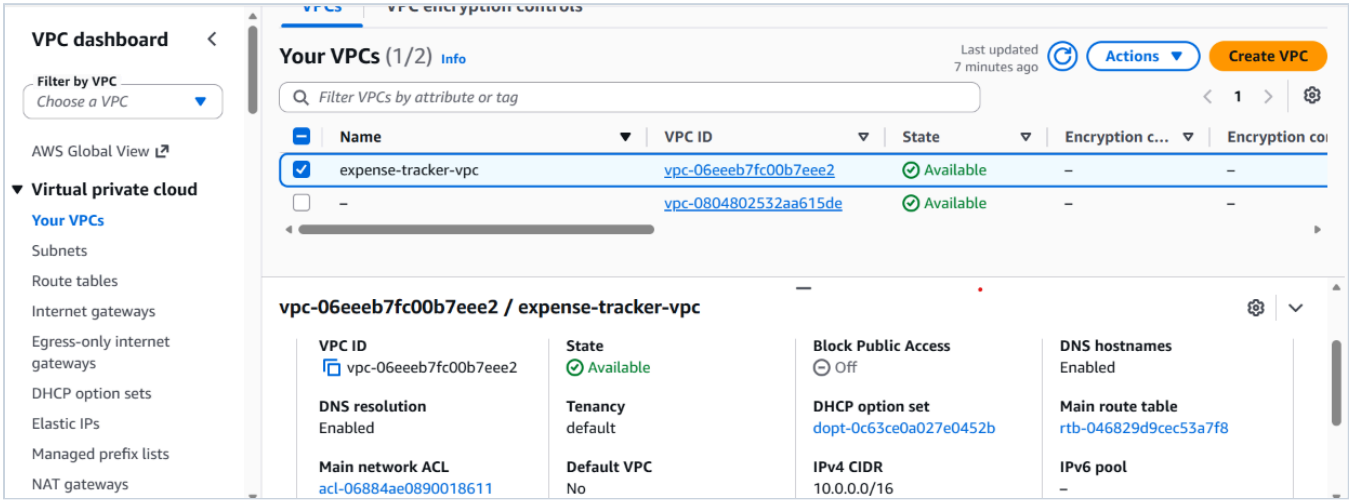


Figure A1 - VPC `expense-tracker-vpc`, IPv4 CIDR 10.0.0.0/16.

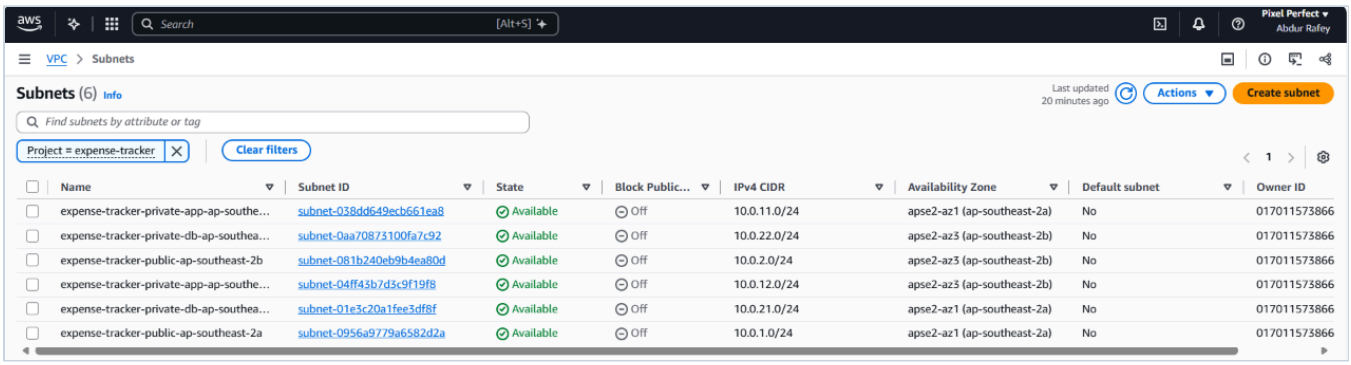


Figure A2 - Six subnets: public, private-app and private-DB in two Availability Zones.

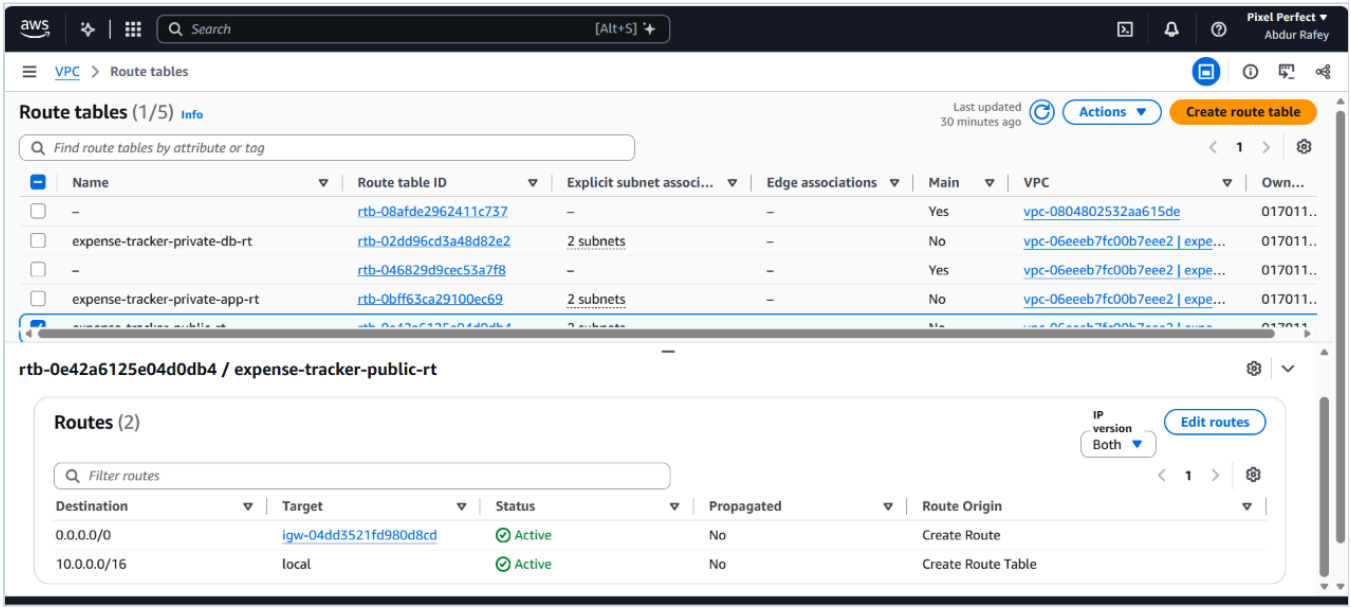


Figure A3 - Public route table: 0.0.0.0/0 → Internet Gateway.

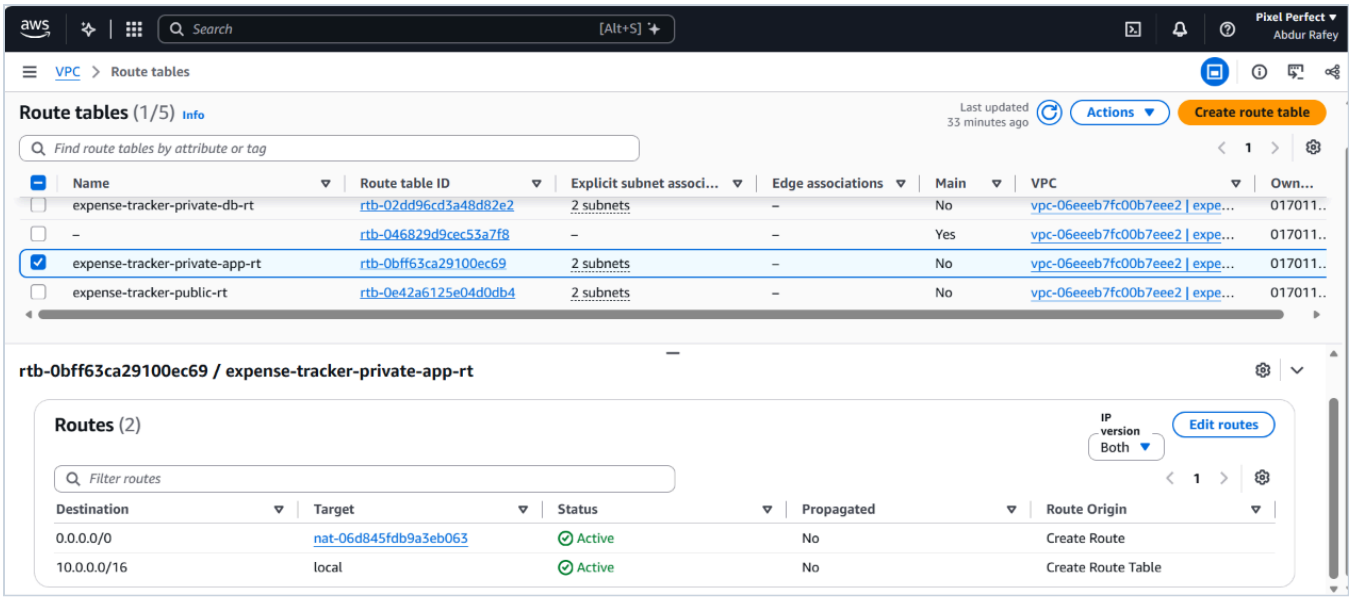


Figure A4 - Private-app route table: 0.0.0.0/0 → NAT Gateway (outbound only).

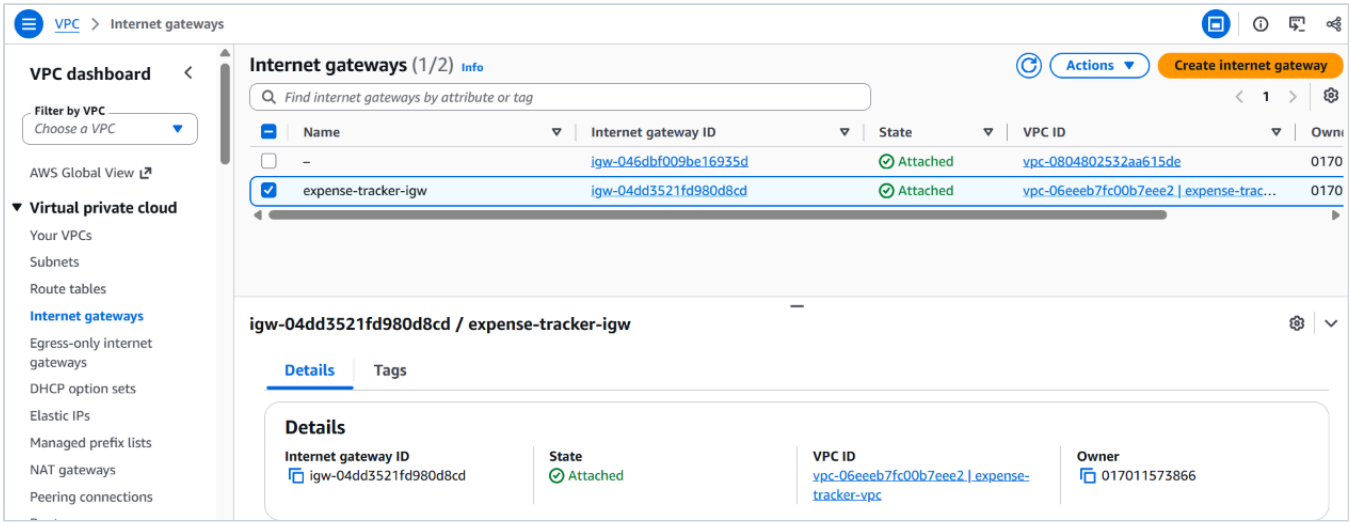


Figure A5 - Internet Gateway attached to the VPC.

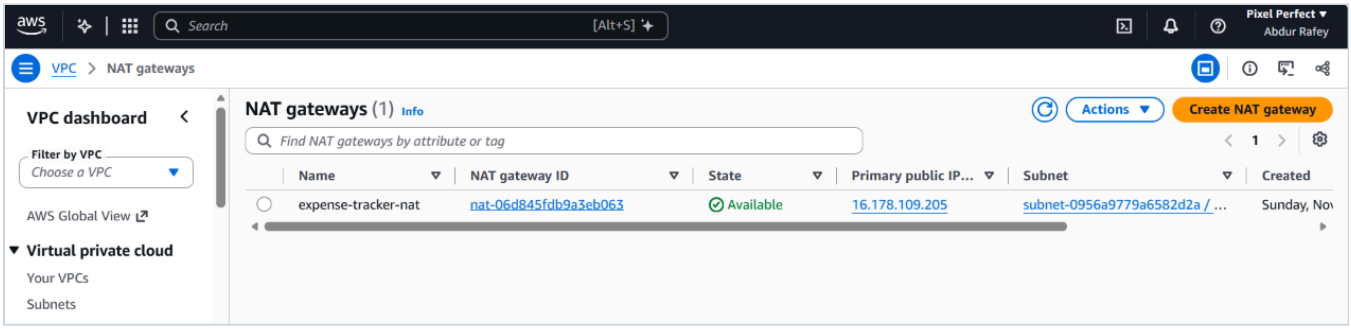


Figure A6 - NAT Gateway (Available) with its public Elastic IP.

A.2 Compute instances / container hosts

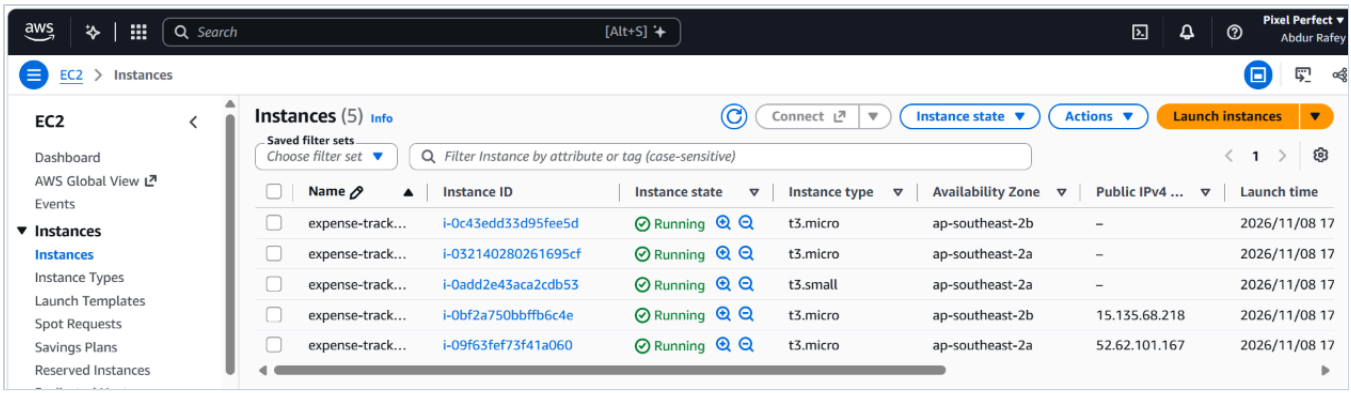


Figure A7 - Five running instances: 2 web (public IPs), 2 app and 1 MongoDB host (no public IPs).

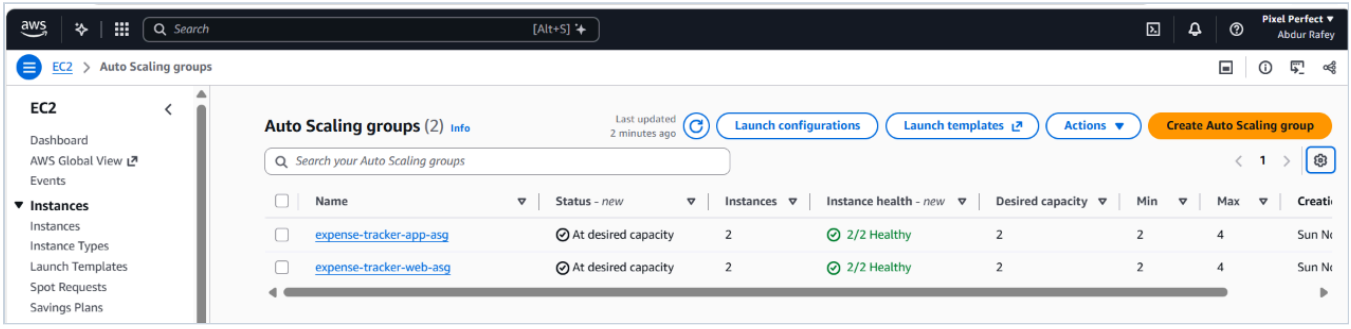


Figure A8 - Web and app Auto Scaling Groups: 2/2 healthy, min 2, max 4.

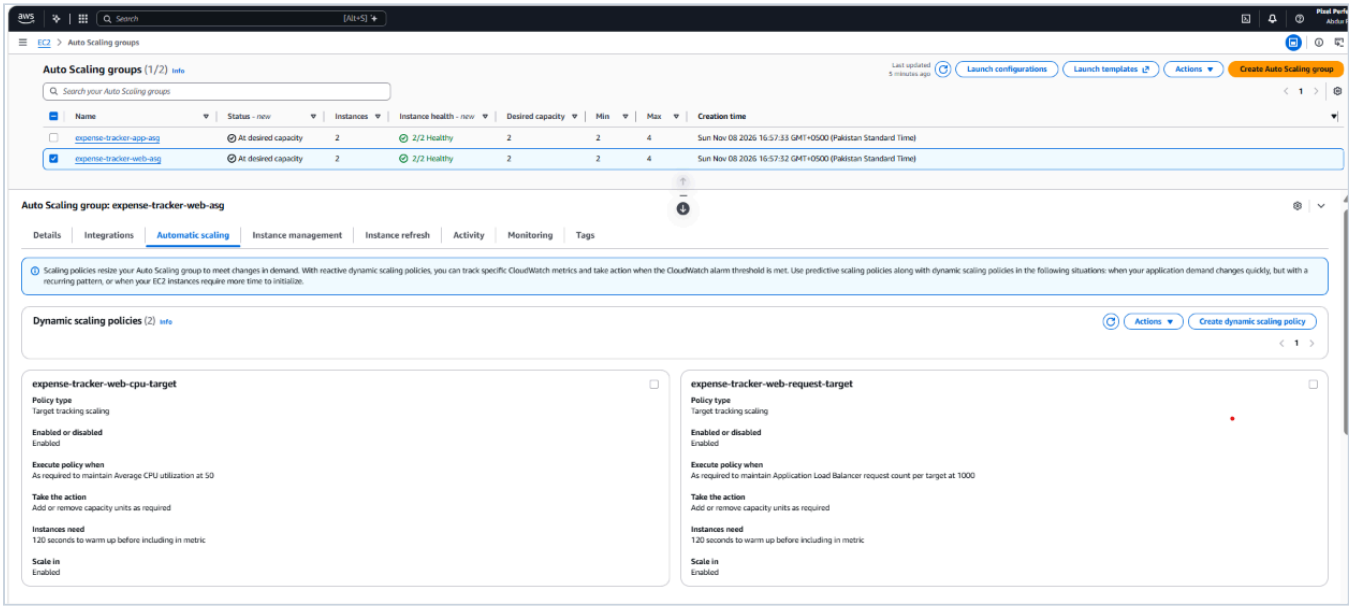


Figure A9 - Target-tracking scaling policies: CPU 50 % and 1,000 requests per target.

A.3 Security groups / firewall rules

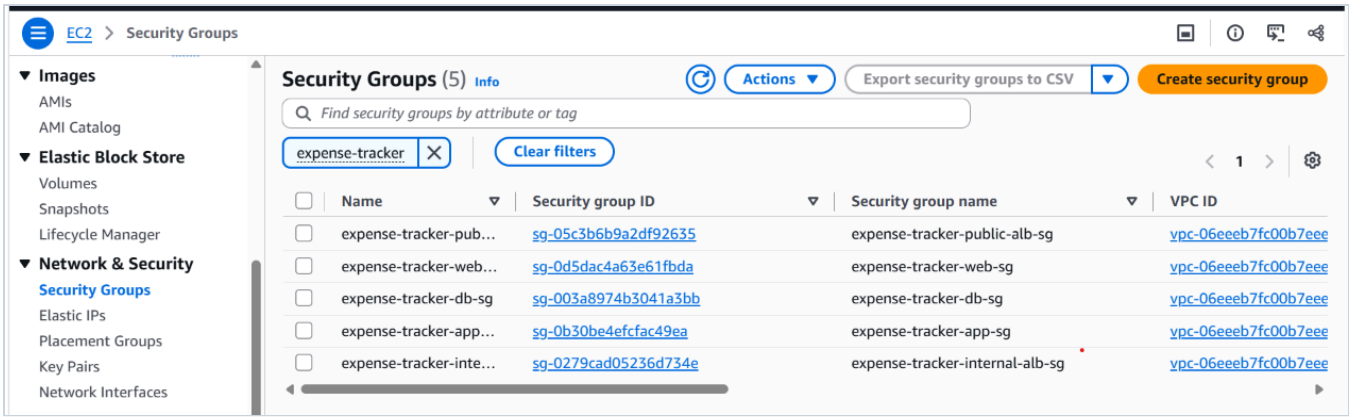


Figure A10 - One security group per tier.

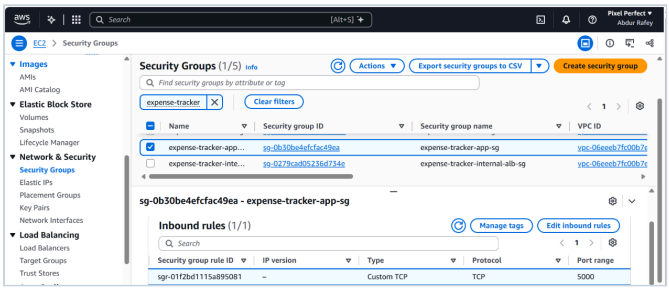


Figure A11 - App SG: inbound TCP 5000 only (source: internal-ALB SG).

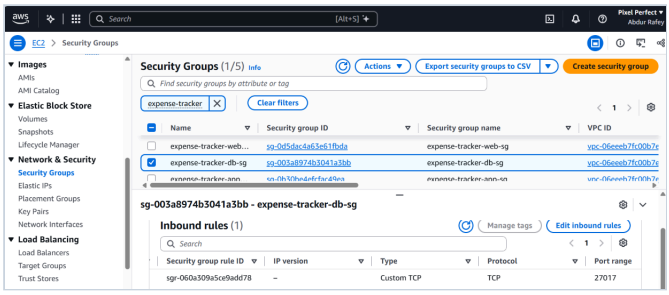


Figure A12 - DB SG: inbound TCP 27017 only (source: app SG).

A.4 Load balancer configuration

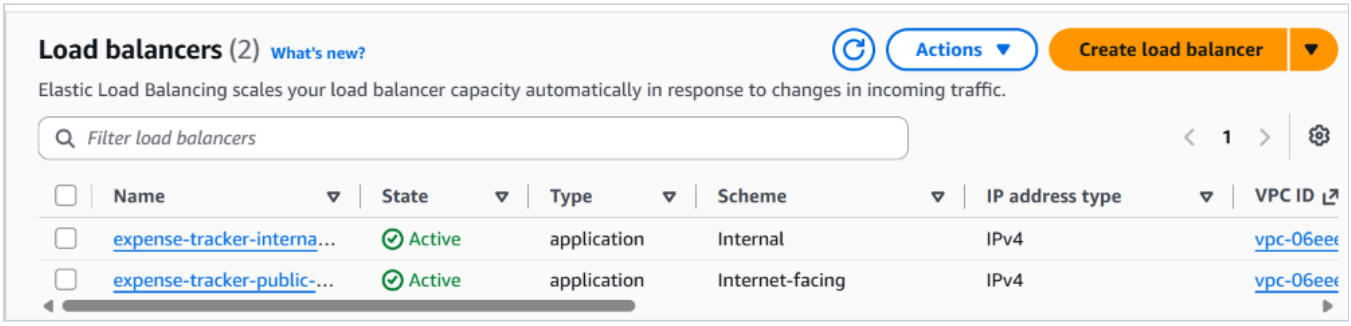


Figure A13 - Internet-facing public ALB and internal ALB, both Active.

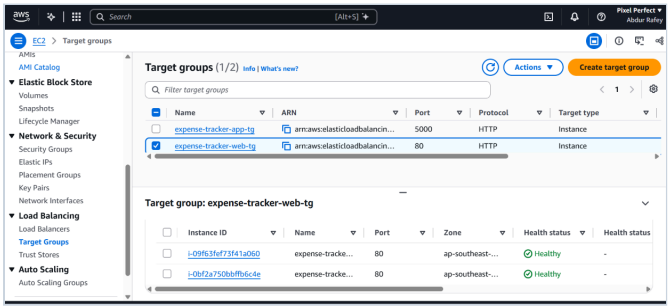


Figure A14 - Web target group: 2 healthy targets on port 80.

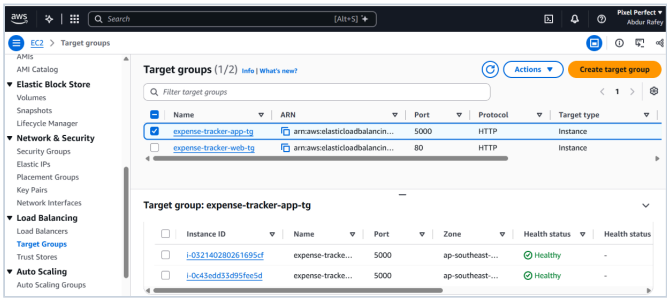


Figure A15 - App target group: 2 healthy targets on port 5000.

A.5 Database instance

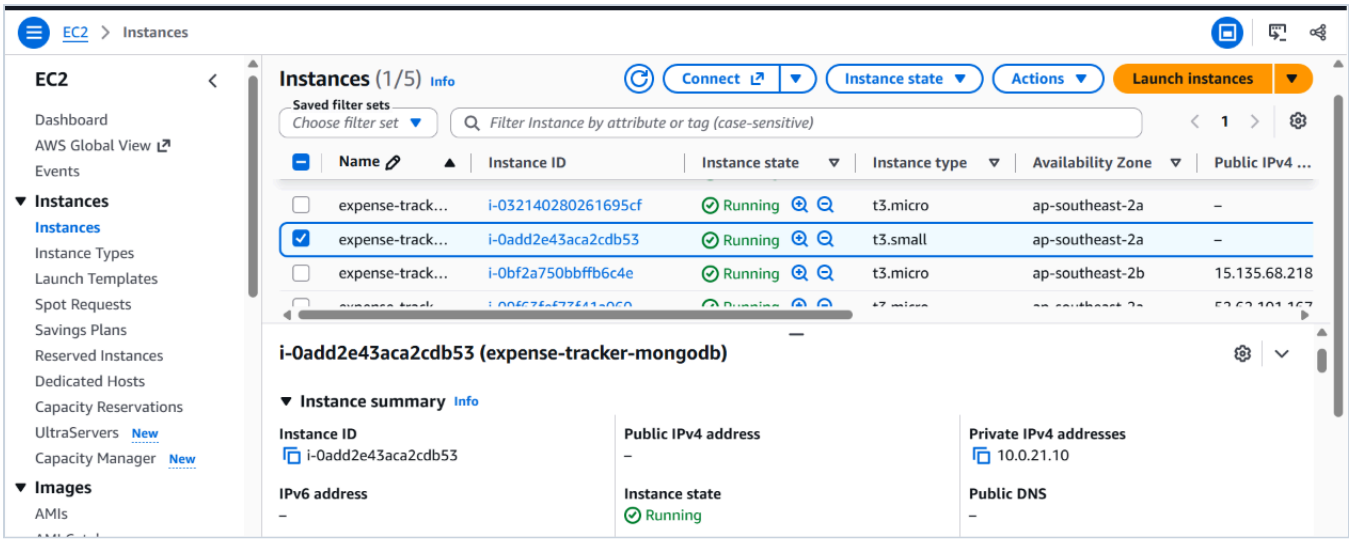


Figure A16 - MongoDB host `expense-tracker-mongodb` : private IP 10.0.21.10, no public IPv4.

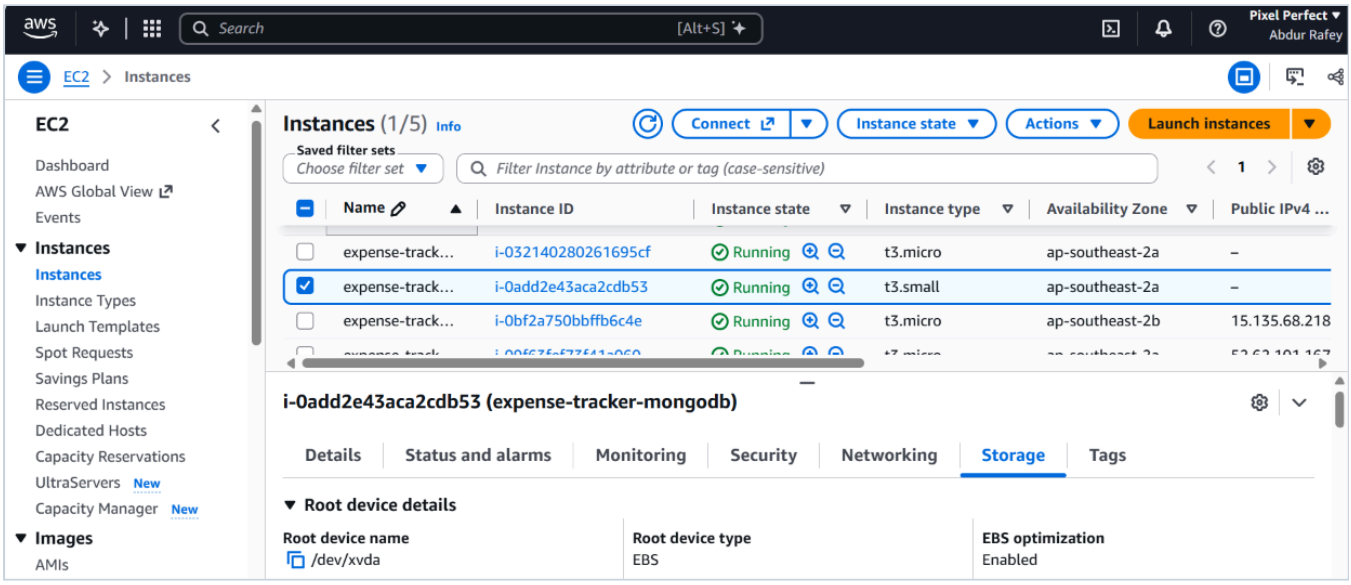


Figure A17 - Database host storage (EBS); the separate 20 GiB encrypted data volume is listed in `docs/evidence/05-database.txt` .

A.6 Docker images and container registry

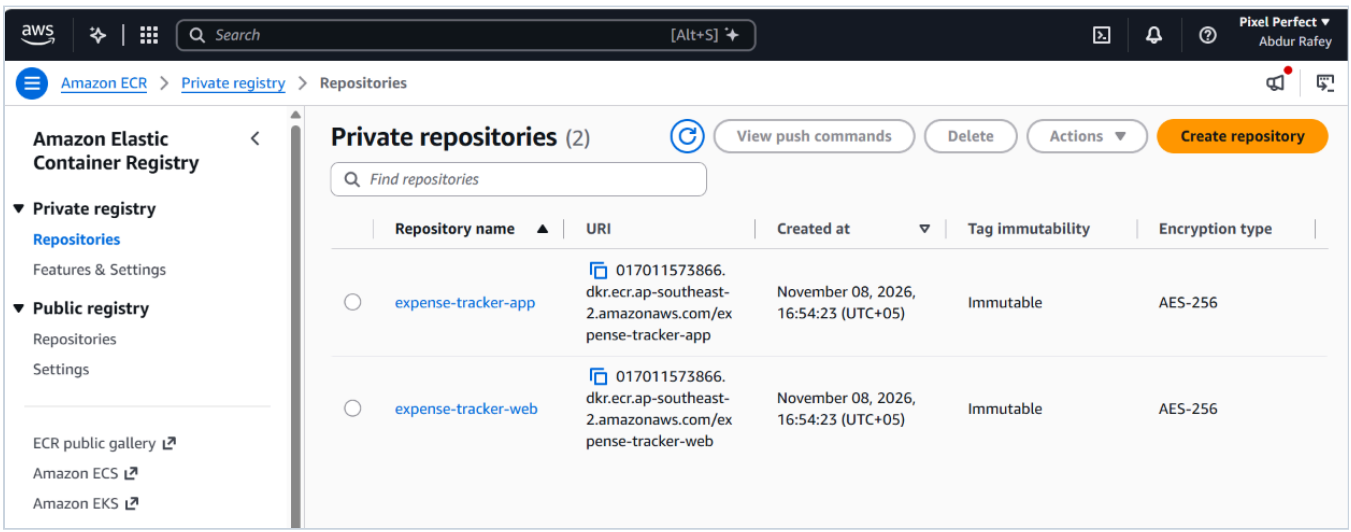


Figure A18 - Amazon ECR private repositories (immutable tags, AES-256).

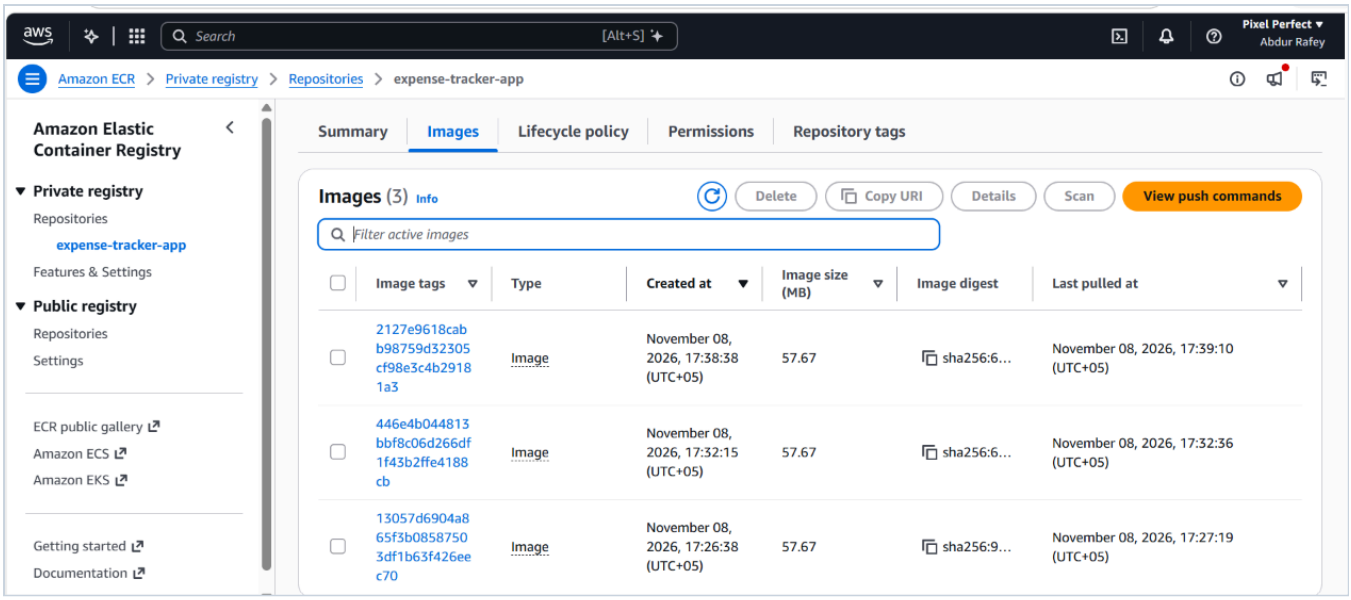


Figure A19 - Images pushed by the pipeline, tagged with git commit SHAs.

A.7 CI/CD pipeline run

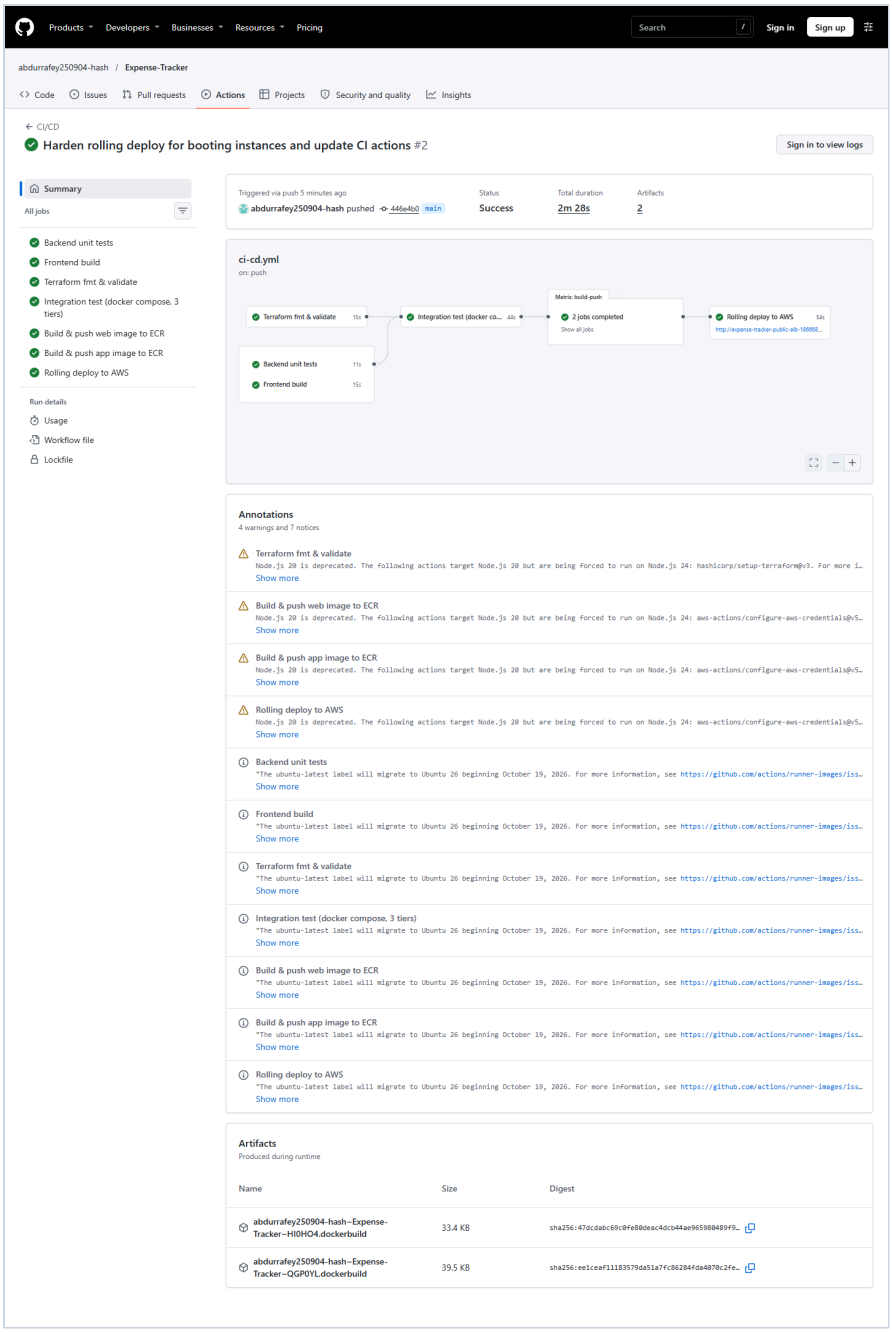


Figure A20 - Successful GitHub Actions run: unit tests, frontend build, Terraform validate → integration test → build & push (matrix) → rolling deploy to AWS. Build-and-push logs: `docs/evidence/06c-docker-build-log-*.txt`.

A.8 Running application accessed via a browser

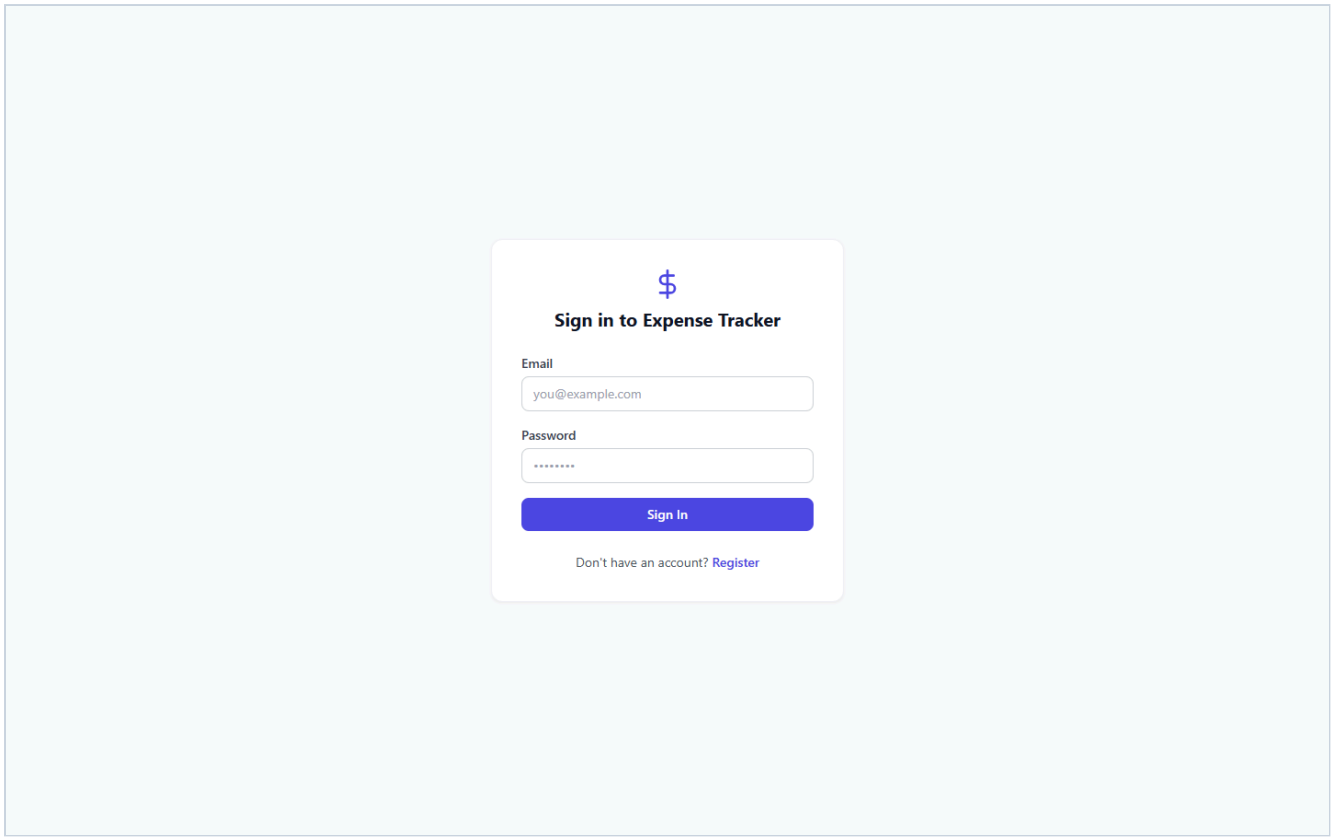


Figure A21 - Login page served through the public load balancer.

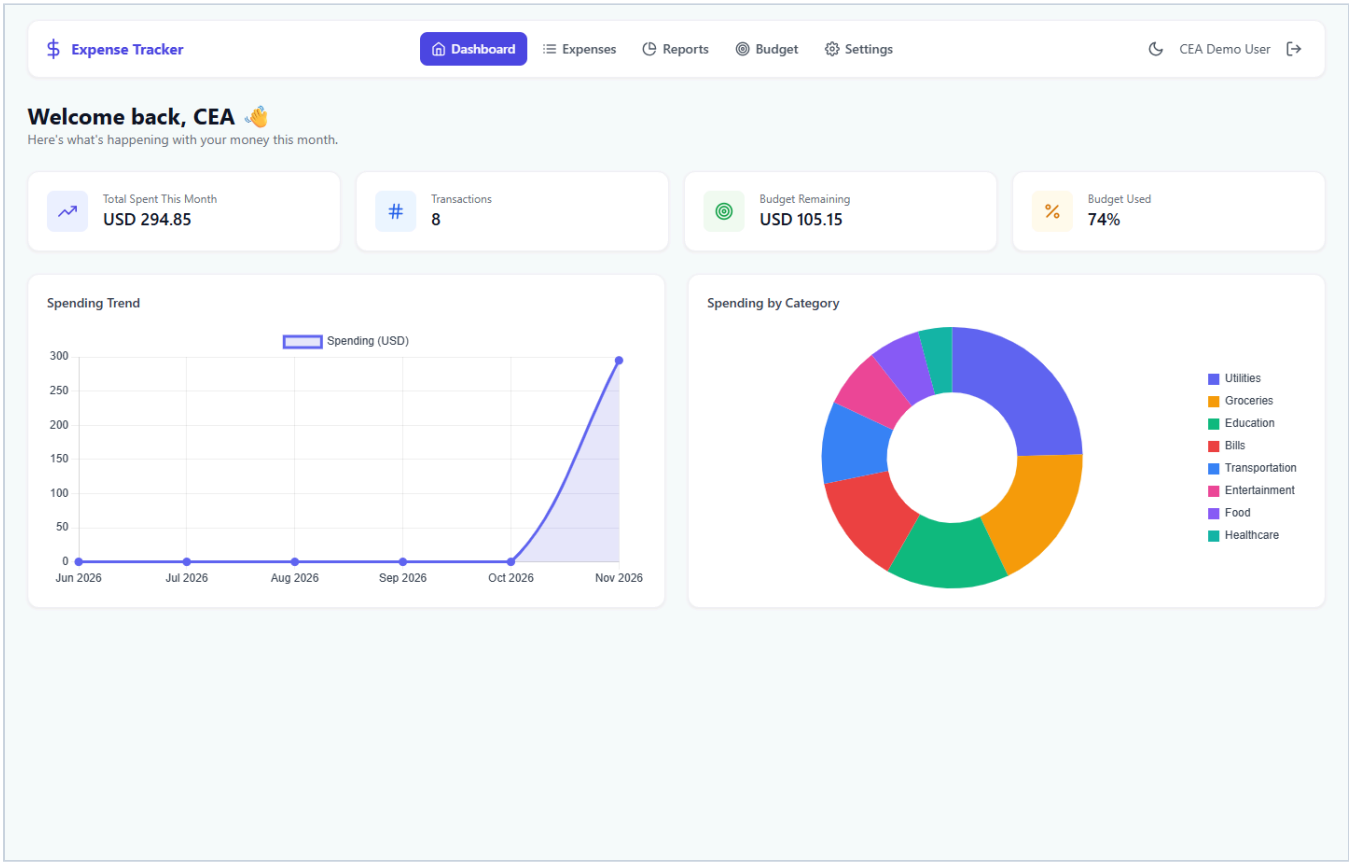


Figure A22 - Dashboard with live data from MongoDB (totals, budget usage, trend and category charts).

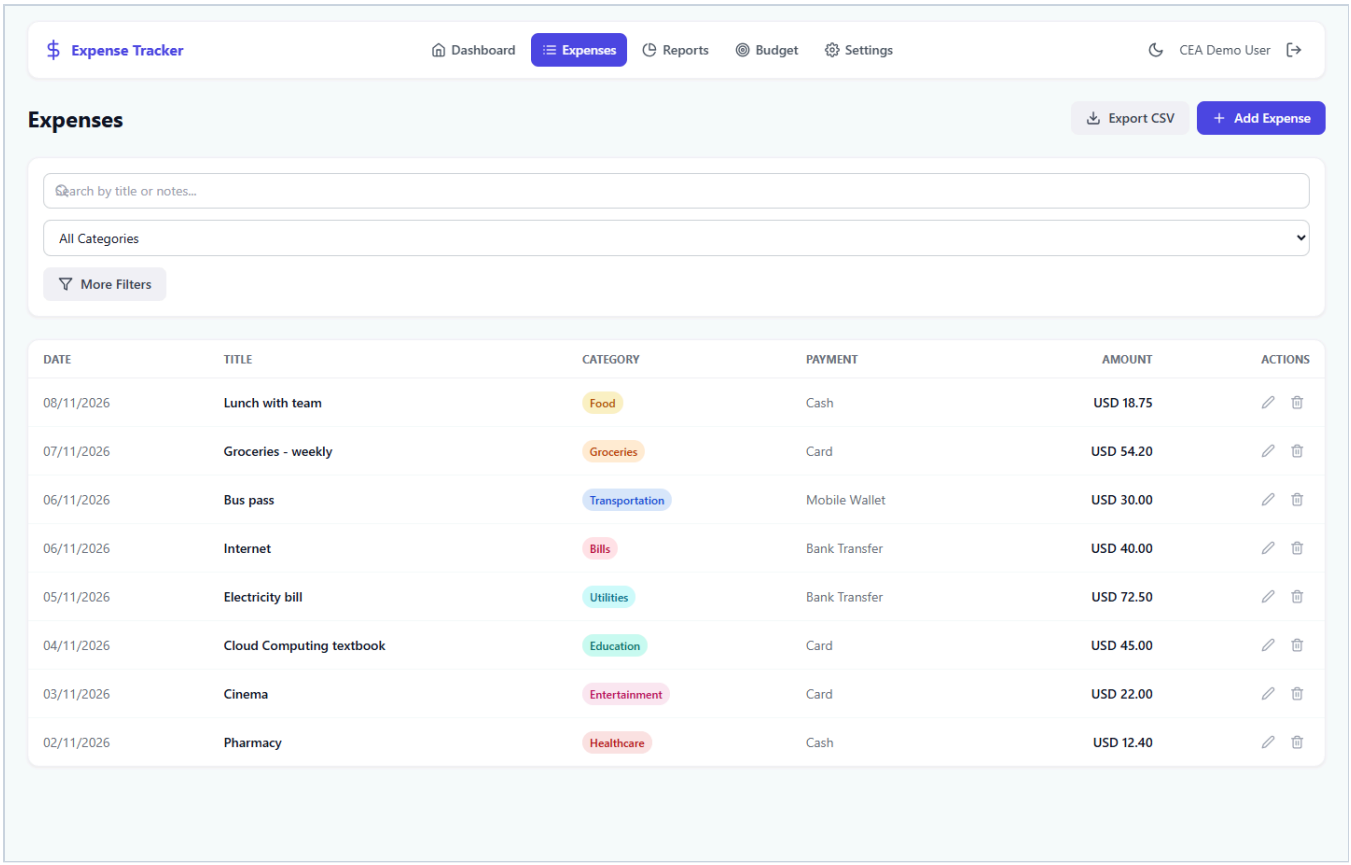


Figure A23 - Expenses list with filtering and pagination.

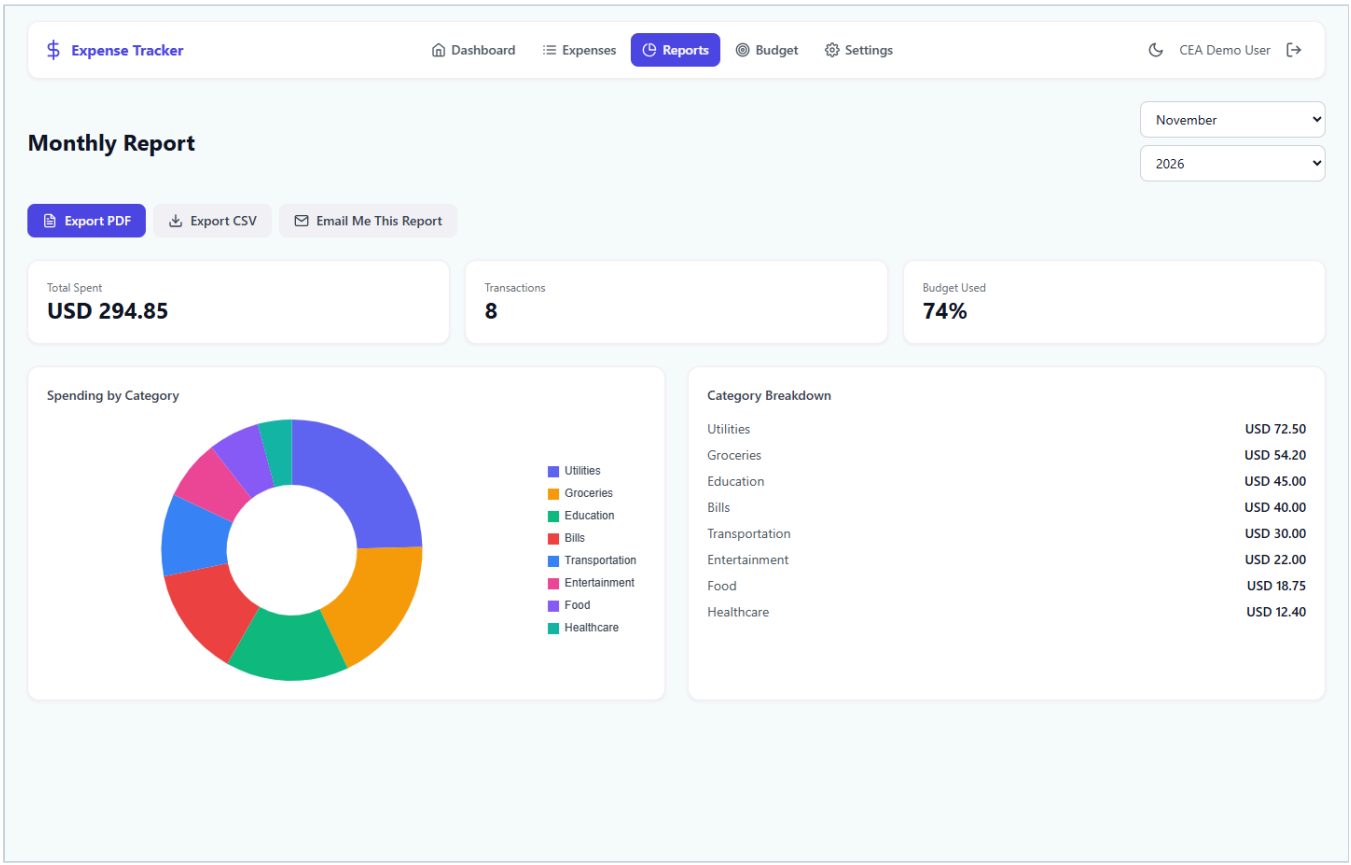


Figure A24 - Monthly report page.

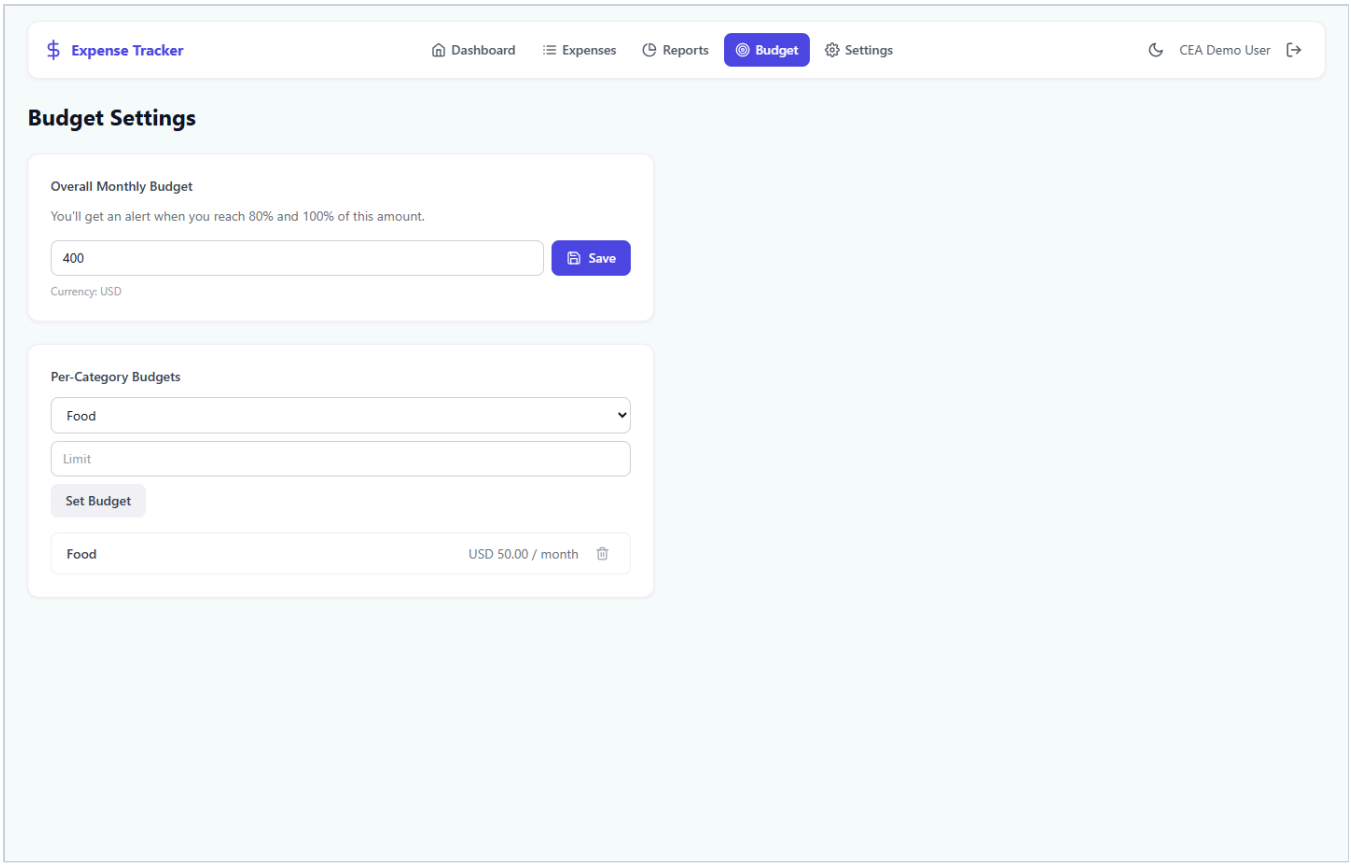


Figure A25 - Budget page with category budget and alerts.

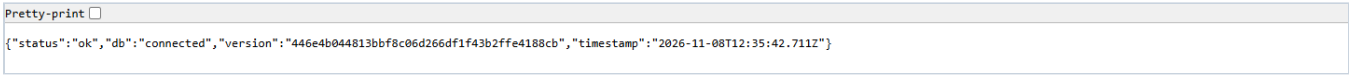


Figure A26 - `/api/health` through all tiers: deployed version and `db: connected`.

Appendix B - Source Files and Repository

Repository: https://github.com/abdurrafey250904-hash/Expense-Tracker

File	Purpose
`frontend/Dockerfile`	Web-tier image (Vite build → unprivileged Nginx)
`frontend/nginx/default.conf.template`	Nginx: SPA routing, `/healthz`, security headers, `/api` reverse proxy
`backend/Dockerfile`	Application-tier image (Express API, non-root, health check)
`docker-compose.yml`	Local / CI orchestration of web, app and MongoDB
`.github/workflows/ci-cd.yml`	CI/CD pipeline definition
`.github/scripts/rolling-deploy.sh`	Rolling deployment through SSM Run Command
`infra/terraform/*.tf`	Infrastructure as code: VPC, security groups, ALBs, ASGs, database, ECR, IAM, secrets, monitoring
`infra/terraform/user_data*.sh.tftpl`	Instance bootstrap: Docker, deploy service, MongoDB volume and backups
`backend/tests/app.test.js`	Unit tests run by the pipeline
`scripts/collect-evidence.sh`	Exports AWS CLI evidence to `docs/evidence/`
`DEPLOYMENT.md`	Step-by-step deployment log with one-line explanations

Kubernetes manifests are not applicable: the Kubernetes enhancement was not selected (the brief requires at least one enhancement; five others were implemented).